

GovSight Financial Analyzer

Security Audit Report

Date: June 23, 2025

Branch: Dev

Repository: FundingScenarioAnalyzer

Scope: Complete codebase security analysis

CRITICAL VULNERABILITIES FOUND

HIGH RISK - SQL Injection Vulnerabilities

1. Direct String Interpolation in SQL Queries

Files: db_connection.py, app.py, ai_assistant_clean.py, streamlit_multiorg_frontend.py

Critical Issues Found:

- cursor.execute(f"SELECT name FROM sqlite_master WHERE type='table' AND name='{table}'")
- f"SELECT * FROM DepartmentPerformance WHERE Organization = '{org_name}'"
- query = f"SELECT * FROM {selected_table}"

Risk Level: CRITICAL

Impact: Full database compromise, data exfiltration, unauthorized access

2. AI-Generated SQL Execution

Files: attached_assets/govsight_ai_bot.py, attached_assets/govsight_portal.py

Critical Issues:

- Direct execution of AI-generated SQL without validation
- sql_query = response['choices'][0]['message']['content'].strip()
- No sanitization before execution

Risk Level: CRITICAL

Impact: Arbitrary SQL execution through AI manipulation

MEDIUM RISK VULNERABILITIES

Authentication & Session Management

1. Weak Password Storage

File: admin_panel.py, users_table.csv

Issues:

- Passwords stored in plain text in CSV file
- Default password "govsight123" is hardcoded
- No password hashing implementation
- Session state not properly secured

2. Insufficient Access Control

Files: admin_panel.py, db_connection.py

Issues:

- Role-based access not consistently enforced
- Department filtering bypassable
- Admin panel accessible without proper validation

Input Validation Issues

1. Unvalidated User Inputs

Files: admin_panel.py, ai_assistant_clean.py

Issues:

- Multiple st.text_input without validation
- username = st.text_input("Username", value="admin_user")
- password = st.text_input("Password", type="password")

2. File Upload Vulnerabilities

Files: ai_assistant_clean.py, admin_panel.py

Issues:

- No file type validation
- No file size limits
- No malware scanning

IMMEDIATE SECURITY FIXES REQUIRED

Priority 1: SQL Injection Prevention

Replace ALL f-string SQL queries with parameterized queries:

BEFORE (VULNERABLE):

```
cursor.execute(f"SELECT * FROM {table} WHERE org = '{org}'")
```

AFTER (SECURE):

```
cursor.execute("SELECT * FROM ? WHERE org = ?", (table, org))
```

Priority 2: Password Security

Implement password hashing:

```
import hashlib
```

```
import secrets
```

```
def hash_password(password):
```

```
    salt = secrets.token_hex(16)
```

```
    password_hash = hashlib.pbkdf2_hmac('sha256',
```

```
                                       password.encode(),
```

```
                                       salt.encode(),
```

```
                                       100000)
```

```
    return salt + password_hash.hex()
```

Priority 3: Input Validation

Add input sanitization:

```
import re
```

```
def sanitize_input(user_input):
```

```
    dangerous_patterns = [',', '--', '/', '*', 'xp_', 'sp_']
```

```
for pattern in dangerous_patterns:
    user_input = user_input.replace(pattern, "")
return user_input.strip()
```

SECURITY RECOMMENDATIONS

Immediate Actions (Week 1)

1. Replace all f-string SQL queries with parameterized queries
2. Implement password hashing for all user accounts
3. Add input validation to all user inputs
4. Remove AI SQL generation features or add strict validation

Short Term (Month 1)

1. Implement proper session management with secure tokens
2. Add role-based access control enforcement
3. Implement file upload validation with type/size restrictions
4. Add security logging for audit trails

Long Term (Quarter 1)

1. Implement database encryption at rest
2. Add API rate limiting to prevent abuse
3. Implement security headers for web interface
4. Regular security testing and penetration testing

COMPLIANCE CONSIDERATIONS

Municipal Data Protection

- Implement FOIA compliance logging
- Add data retention policies
- Ensure audit trail completeness

Financial Data Security

- PCI compliance considerations for payment data
- SOX compliance for financial reporting
- State/local government security standards

CONCLUSION

Current Security Status: HIGH RISK

Critical Vulnerabilities: 5

Medium Risk Issues: 8

Estimated Fix Time: 2-3 weeks for critical issues

The GovSight Financial Analyzer contains several critical security vulnerabilities, primarily SQL injection risks from unparameterized queries. Immediate action is required to secure the application before production deployment.

Next Steps:

1. Begin SQL injection fixes immediately
2. Implement password hashing system
3. Add comprehensive input validation
4. Schedule security code review

This audit should be repeated after fixes are implemented.